OOPS with Java
Mid-Semester Solutions

March 2026 Paper

Contents

1 Section A **1**
- 1.1 Q1. Define the fundamental principles of Object-Oriented Programming. 1
- 1.2 Q2. Explain the difference between == operator and equals() method in Java with a small example. 2
- 1.3 Q3. Explain diamond problem in Java. How can it be resolved? 2
- 1.4 Q4. Differentiate between an abstract class and an interface in Java. 3
- 1.5 Q5. Write down all the errors associated with this code. 4

2 Section B **6**
- 2.1 Q6. Design a Java program to calculate and display the grade of a student. 6
- 2.2 Q7. Write a Java program to demonstrate the use of this, super and final keywords with suitable example. 8
- 2.3 Q8 (Option A). User-defined exception: InvalidAgeException 9
- 2.4 Q8 (Option B). Types of nested classes in Java and a demo program 10

3 Section C **11**
- 3.1 Q9 (Option A). Bank Management System 11
- 3.2 Q9 (Option B). Library Management System 14

4 Quick Revision Table **16**

Note

These solutions are prepared from the mid-semester question paper images available in the subject folder. For the OR questions, both alternatives are solved so the document is useful as a full study resource.

1 Section A

1.1 Q1. Define the fundamental principles of Object-Oriented Programming.

Exam-ready answer: The four fundamental principles of OOP are **encapsulation**, **abstraction**, **inheritance**, and **polymorphism**. They help organize data and behavior together, hide

unnecessary details, reuse code, and allow the same interface to show different behaviors.

Detailed explanation:

- **Encapsulation:** Data and related methods are kept together inside a class. Access is controlled using modifiers such as `private`, `protected`, and `public`.
- **Abstraction:** Only essential details are exposed to the user. Internal implementation is hidden. Abstract classes and interfaces support abstraction in Java.
- **Inheritance:** A new class can reuse properties and methods of an existing class. This improves code reuse and models "is-a" relationships.
- **Polymorphism:** The same method name or interface can behave differently depending on the object. In Java this is commonly seen through method overloading and method overriding.

Small example:

```java
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Here, the class groups data and behavior, `Dog` inherits from `Animal`, and overriding shows polymorphism.

1.2 Q2. Explain the difference between == operator and equals() method in Java with a small example.

Exam-ready answer: In Java, `==` compares references for objects, while `equals()` compares logical content if the class overrides it. For primitive types, `==` compares values directly.

Detailed explanation:

- For **primitive data types**, `==` checks actual value equality.
- For **objects**, `==` checks whether two references point to the same memory location.
- `equals()` is a method from `Object`. Classes such as `String` override it to compare actual content.

Example:

```java
public class EqualsDemo {
    public static void main(String[] args) {
        String a = new String("Java");
```

```java
        String b = new String("Java");

        System.out.println(a == b); // false
        System.out.println(a.equals(b)); // true
    }
}
```

Why?

- `a == b` is `false` because `a` and `b` are two different objects.
- `a.equals(b)` is `true` because both strings store the same sequence of characters.

1.3 Q3. Explain diamond problem in Java. How can it be resolved?

Exam-ready answer: The diamond problem happens when a class inherits the same method implementation through two different paths, creating ambiguity about which version should be used. Java avoids this for classes by disallowing multiple inheritance of classes.

Detailed explanation:

- In languages that allow multiple inheritance of classes, a child class may inherit the same method from two parent classes.
- This creates confusion: which parent implementation should be used?
- Java resolves this by **not allowing multiple inheritance of classes**.
- Java **does allow multiple inheritance through interfaces**. If two interfaces provide the same `default` method, the implementing class must override it explicitly.

Interface example:

```java
interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B {
    default void show() {
        System.out.println("B");
    }
}

class C implements A, B {
    @Override
    public void show() {
        System.out.println("Resolved in C");
    }
}
```

Conclusion: Java resolves the diamond problem by avoiding multiple inheritance of classes and forcing explicit resolution when conflicting interface default methods exist.

1.4 Q4. Differentiate between an abstract class and an interface in Java.

Exam-ready answer:

Point	Abstract Class	Interface
Purpose	Partial abstraction and common base behavior	Pure contract for behavior
Methods	Can have abstract and concrete methods	Mostly abstract methods; can also have default and static methods
Variables	Can have instance variables	Variables are public, static, and final by default
Constructors	Allowed	Not allowed
Inheritance	A class can extend only one abstract class	A class can implement multiple interfaces
Use case	Shared state and partial implementation	Common capability or role across unrelated classes

Example idea:

- Use an **abstract class** when classes share state and some common implementation.
- Use an **interface** when different classes only need to follow the same contract.

1.5 Q5. Write down all the errors associated with this code.

Question code:

```java
class Outer {
    private int x = 10;

    static class StaticNested {
        void display() {
            System.out.println(x);
        }
    }

    class Inner {
        static int y = 20;
        void show() {
            System.out.println("Inner y = " + y);
        }
    }

    public static void main(String[] args) {
        Outer.Inner obj1 = new Outer.Inner();
        StaticNested obj2 = new StaticNested();
        obj2.display();
        Inner obj3 = new Inner();
        obj3.show();
        Outer o = null;
        o.new Inner();
```

```
        Outer.StaticNested obj4 = new Outer.StaticNested();
        obj4.show();
    }
}
```

Error-by-error analysis:

1. **Illegal access of non-static member from static nested class**
   - `x` is an instance variable of `Outer`.
   - `StaticNested` is a static nested class.
   - Therefore `System.out.println(x);` causes a compile-time error.
2. **Illegal static member in a non-static inner class**
   - `Inner` is a non-static inner class.
   - A non-static inner class cannot declare a static member unless it is a constant (`static final` compile-time constant).
   - So `static int y = 20;` causes a compile-time error.
3. **Wrong way to create an inner class object**
   - `Outer.Inner obj1 = new Outer.Inner();` is invalid.
   - A non-static inner class must be created with an outer object, for example:

     ```
     Outer o = new Outer();
     Outer.Inner obj1 = o.new Inner();
     ```
4. **Wrong creation of inner object inside static method**
   - `Inner obj3 = new Inner();` is invalid inside `main()` because there is no enclosing `Outer` object.
5. **Runtime error due to null outer reference**
   - `Outer o = null;` followed by `o.new Inner();` throws a `NullPointerException` at runtime because `o` is null.
6. **Calling a non-existing method**
   - `obj4` is of type `StaticNested`.
   - `StaticNested` has method `display()`, not `show()`.
   - So `obj4.show();` causes a compile-time error.

Corrected version:

```java
class Outer {
    private int x = 10;

    static class StaticNested {
        void display(Outer obj) {
            System.out.println(obj.x);
        }
    }

    class Inner {
        int y = 20;
        void show() {
            System.out.println("Inner y = " + y);
        }
    }

    public static void main(String[] args) {
        Outer o = new Outer();
        Outer.Inner obj1 = o.new Inner();
        StaticNested obj2 = new StaticNested();

        obj2.display(o);
        obj1.show();

        Outer.StaticNested obj4 = new Outer.StaticNested();
        obj4.display(o);
    }
}
```

2 Section B

2.1 Q6. Design a Java program to calculate and display the grade of a student.

What the question wants:

- A `Student` class
- Data members such as roll number, name, marks array, total, percentage, grade
- Methods to accept details, calculate total and percentage, assign grade, and display the result

Design choice used here:

- Percentage >= 90: A
- Percentage >= 80: B
- Percentage >= 70: C
- Percentage >= 60: D
- Otherwise: F

Solution code:

```java
import java.util.Scanner;

class Student {
    private int rollNumber;
    private String name;
    private int[] marks;
    private int total;
    private double percentage;
    private char grade;

    Student(int subjects) {
        marks = new int[subjects];
    }

    void acceptDetails() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter roll number: ");
        rollNumber = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter name: ");
        name = sc.nextLine();

        System.out.println("Enter marks for 5 subjects:");
        for (int i = 0; i < marks.length; i++) {
            System.out.print("Subject " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
        }
    }

    void calculateResult() {
        total = 0;
        for (int mark : marks) {
            total += mark;
        }

        percentage = total / (double) marks.length;

        if (percentage >= 90) {
            grade = 'A';
        } else if (percentage >= 80) {
            grade = 'B';
        } else if (percentage >= 70) {
            grade = 'C';
        } else if (percentage >= 60) {
            grade = 'D';
        } else {
            grade = 'F';
        }
    }
```

```java
    void displayResult() {
        System.out.println("\n----- Student Result -----");
        System.out.println("Roll Number : " + rollNumber);
        System.out.println("Name : " + name);
        System.out.println("Total : " + total);
        System.out.println("Percentage : " + percentage);
        System.out.println("Grade : " + grade);
    }
}

public class StudentGradeDemo {
    public static void main(String[] args) {
        Student s = new Student(5);
        s.acceptDetails();
        s.calculateResult();
        s.displayResult();
    }
}
```

Sample run:

```
Enter roll number: 101
Enter name: Aditi
Enter marks for 5 subjects:
Subject 1: 88
Subject 2: 91
Subject 3: 84
Subject 4: 79
Subject 5: 90

----- Student Result -----
Roll Number : 101
Name : Aditi
Total : 432
Percentage : 86.4
Grade : B
```

Explanation:

- The constructor creates the marks array for 5 subjects.
- `acceptDetails()` stores student data.
- `calculateResult()` computes total, percentage, and grade.
- `displayResult()` prints the final result in readable form.

2.2 Q7. Write a Java program to demonstrate the use of this, super and final keywords with suitable example.

Concept summary:

- **this** refers to the current object.

- **super** refers to the immediate parent class object.
- **final** is used to stop modification, overriding, or inheritance depending on context.

Solution code:

```java
class Person {
    protected String name;

    Person(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Person name: " + name);
    }
}

class Student extends Person {
    private final int rollNo;

    Student(String name, int rollNo) {
        super(name); // use of super
        this.rollNo = rollNo; // use of this
    }

    @Override
    void display() {
        super.display(); // call parent version
        System.out.println("Student roll no: " + this.rollNo);
    }

    final void examRule() {
        System.out.println("Final method: exam rule cannot be overridden.");
    }
}

final class Utility {
    static void message() {
        System.out.println("Final class cannot be inherited.");
    }
}

public class KeywordDemo {
    public static void main(String[] args) {
        Student s = new Student("Riya", 23);
        s.display();
        s.examRule();
        Utility.message();
    }
}
```

How each keyword is demonstrated:

- `this.name = name;` distinguishes instance variable from constructor parameter.
- `super(name);` calls the parent constructor.
- `super.display();` calls the parent method from the child class.
- `final int rollNo;` means the roll number cannot be reassigned.
- `final void examRule()` means child classes cannot override that method.
- `final class Utility` means no class can extend `Utility`.

Sample output:

```
Person name: Riya
Student roll no: 23
Final method: exam rule cannot be overridden.
Final class cannot be inherited.
```

2.3 Q8 (Option A). User-defined exception: InvalidAgeException

Solution code:

```java
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

public class AgeValidator {
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age is less than 18. Not eligible.");
        }
        System.out.println("Age is valid. Eligible.");
    }

    public static void main(String[] args) {
        int age = 16;

        try {
            System.out.println("Validation starts");
            validateAge(age);
            System.out.println("Validation completed successfully");
        } catch (InvalidAgeException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }

        System.out.println("Program continues after try-catch");
    }
}
```

Flow of execution:

1. `main()` starts and enters the `try` block.
2. `validateAge(age)` is called.
3. Since `age < 18`, an `InvalidAgeException` object is created and thrown.
4. Control immediately leaves `validateAge()` and goes to the matching `catch` block.
5. The exception message is printed.
6. After the catch block, the program continues normally.

Sample output:

```
Validation starts
Exception caught: Age is less than 18. Not eligible.
Program continues after try-catch
```

2.4 Q8 (Option B). Types of nested classes in Java and a demo program

Types of nested classes in Java:

1. **Static nested class** - declared with `static` inside another class.
2. **Inner class / member inner class** - non-static class inside another class.
3. **Local inner class** - declared inside a method.
4. **Anonymous inner class** - class without a name, usually created for one-time use.

Required demo: static nested class and non-static inner class

```java
class Outer {
    private int x = 100;
    static int z = 500;

    static class StaticNested {
        void display() {
            System.out.println("Static nested class can access z = " + z);
        }
    }

    class Inner {
        void show() {
            System.out.println("Inner class can access x = " + x);
            System.out.println("Inner class can also access z = " + z);
        }
    }
}

public class NestedClassDemo {
    public static void main(String[] args) {
        Outer.StaticNested obj1 = new Outer.StaticNested();
        obj1.display();
```

```
        Outer outer = new Outer();
        Outer.Inner obj2 = outer.new Inner();
        obj2.show();
    }
}
```

Key explanation:

- The `StaticNested` class behaves like a static member of `Outer`. It cannot directly use non-static instance variables such as `x`.
- The `Inner` class belongs to an object of `Outer`, so it can directly access both instance members and static members of the outer class.

Sample output:

```
Static nested class can access z = 500
Inner class can access x = 100
Inner class can also access z = 500
```

3 Section C

3.1 Q9 (Option A). Bank Management System

Required design points covered:

- Class `Account` with data members and methods
- Constructor and method overloading
- Subclass `SavingsAccount` overriding interest calculation
- Interface `Transaction` with `deposit()` and `withdraw()`
- Exception handling for insufficient balance
- Main method to demonstrate all functionalities

Solution code:

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

interface Transaction {
    void deposit(double amount);
    void withdraw(double amount) throws InsufficientBalanceException;
}

class Account implements Transaction {
    protected int accountNumber;
```

```java
    protected String holderName;
    protected double balance;

    Account() {
        this(0, "Unknown", 0.0);
    }

    Account(int accountNumber, String holderName) {
        this(accountNumber, holderName, 0.0);
    }

    Account(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    @Override
    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }

    @Override
    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance for withdrawal");
        }
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
    }

    double calculateInterest() {
        return balance * 0.04;
    }

    void display() {
        System.out.println("Account No : " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance : " + balance);
        System.out.println("Interest : " + calculateInterest());
    }
}

class SavingsAccount extends Account {
    SavingsAccount(int accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }

    @Override
    double calculateInterest() {
        return balance * 0.06;
    }
}
```

```java
public class BankManagementDemo {
    public static void main(String[] args) {
        SavingsAccount acc = new SavingsAccount(101, "Aman", 5000);

        acc.display();
        System.out.println();

        acc.deposit(2000);

        try {
            acc.withdraw(3000);
            acc.withdraw(7000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        System.out.println();
        acc.display();
    }
}
```

Sample output:

```
Account No : 101
Holder Name: Aman
Balance : 5000.0
Interest : 300.0

Deposited: 2000.0
Withdrawn: 3000.0
Exception: Insufficient balance for withdrawal

Account No : 101
Holder Name: Aman
Balance : 4000.0
Interest : 240.0
```

Why this design is correct:

- `Account` stores common data and common transaction behavior.
- Constructor overloading is shown through three constructors.
- `SavingsAccount` overrides interest calculation, showing runtime polymorphism.
- `Transaction` provides a clear contract.
- Insufficient balance is handled through a custom checked exception.

3.2 Q9 (Option B). Library Management System

Required design points covered:

- Classes `Book` and `Member`
- Encapsulation and constructors
- Interface `Issueable` with method `issueBook()`
- Method overloading
- Exception when book is not available
- Main method to demonstrate all functionalities

Solution code:

```
class BookNotAvailableException extends Exception {
    BookNotAvailableException(String message) {
        super(message);
    }
}

interface Issueable {
    void issueBook(Book book) throws BookNotAvailableException;
}

class Book {
    private int bookId;
    private String title;
    private boolean available;

    Book(int bookId, String title) {
        this.bookId = bookId;
        this.title = title;
        this.available = true;
    }

    int getBookId() {
        return bookId;
    }

    String getTitle() {
        return title;
    }

    boolean isAvailable() {
        return available;
    }

    void setAvailable(boolean available) {
        this.available = available;
    }
}

class Member implements Issueable {
    private int memberId;
    private String memberName;
```

```java
    Member(int memberId, String memberName) {
        this.memberId = memberId;
        this.memberName = memberName;
    }

    @Override
    public void issueBook(Book book) throws BookNotAvailableException {
        if (!book.isAvailable()) {
            throw new BookNotAvailableException(
                "Book '" + book.getTitle() + "' is not available"
            );
        }
        book.setAvailable(false);
        System.out.println(memberName + " issued book: " + book.getTitle());
    }

    // Overloaded method
    public void issueBook(Book book, int days) throws BookNotAvailableException {
        issueBook(book);
        System.out.println("Issue duration: " + days + " days");
    }
}

public class LibraryManagementDemo {
    public static void main(String[] args) {
        Book b1 = new Book(1, "Java Fundamentals");
        Member m1 = new Member(101, "Rohit");
        Member m2 = new Member(102, "Sneha");

        try {
            m1.issueBook(b1, 7);
            m2.issueBook(b1);
        } catch (BookNotAvailableException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

Sample output:

```
Rohit issued book: Java Fundamentals
Issue duration: 7 days
Exception: Book 'Java Fundamentals' is not available
```

Why this design is correct:

- `Book` and `Member` use private data members, showing encapsulation.
- Constructors initialize the objects properly.
- `Issueable` makes the issue behavior explicit.
- Overloading is shown by two `issueBook()` methods.

- Exception handling manages the "book not available" case cleanly.

4 Quick Revision Table

Q	**Topic**	**Core idea to remember**
Q1	OOP principles	Encapsulation, abstraction, inheritance, polymorphism
Q2	`==` vs `equals()`	Reference comparison vs content comparison
Q3	Diamond problem	Java avoids multiple inheritance of classes; interfaces need explicit conflict resolution
Q4	Abstract class vs interface	Shared implementation vs contract
Q5	Nested class errors	Static/non-static rules, object creation rules, compile-time vs runtime errors
Q6	Student result system	Class design, array handling, total, percentage, grade logic
Q7	`this`, `super`, `final`	Current object, parent object, restricted modification/override/inheritance
Q8A	Custom exception	Create, throw, catch, and trace program flow
Q8B	Nested classes	Static nested vs member inner class
Q9A	Bank system	Interface, inheritance, overriding, exceptions
Q9B	Library system	Encapsulation, interface, overloading, exception handling